

Pengantar OOP

Case Studi

- Perusahaan A adalah Perusahaan penjualan alat-alat laboratorium rumah sakit yang telah matang secara administrasi dan berkembang pesat. Sistem pencatatan masih menggunakan pendekatan manual. Pimpinan perusahaan menjajaki untuk membentuk tim developer yang terdiri dari seorang lead yang 3 orang developer yang juga merangkap testing, dan implementasi.
- Developer lead sedang merancang arsitektur sistem pengembangan aplikasi yang mana dan pengembangan aplikasi akan dikerjakan secara parallel oleh masing-masing developer.
- Developer lead memikirkan untuk bagaimana tugas pengembangan dapat dibagikan berdasarkan keahlian masing-masing pengembang seperti FrontEnd UI, Business Process, dan Database.
- Developer lead akan banyak berfokus dan Business/System Analyst serta melakukan Manajemen Proyek Sistem Informasi.

Identifikasi Masalah

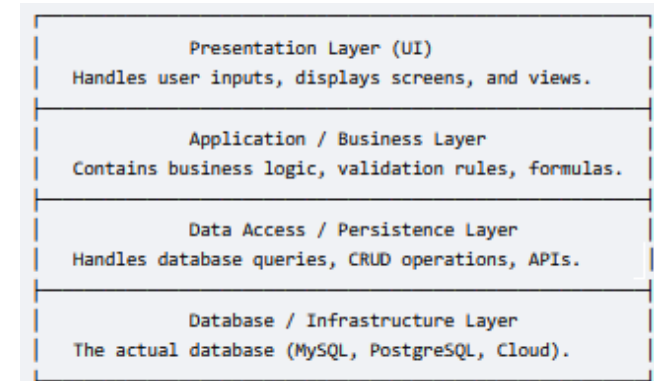
- Bagaimana memindahkan logika bisnis manual ke dalam kode melalui rancangan terstruktur?
- Diagram apa yang dapat membantu komunikasi antara Lead (Business/System Analyst) dengan tim Developer?
- Bagaimana antisipasi aplikasi terhadap perubahan bisnis dan skalabilitas di masa depan dengan menekankan kepada kemudahan pemeliharaan dan pengembangan aplikasi untuk menjawab tantangan jaman.
- Bagaimana memetakan dokumen manual dan entitas business kedalam database?
- Bagaimana pembagian tugas dimana masing-masing developer yang memiliki spesialisasi berbeda (UI, Logika Bisnis dan Database) yang bekerja secara parallel sehingga tidak saling tumpang tindih (overlapping) mengerjakan koding secara berulang dan pendekatan yang berbeda.
- Bagaimana masing-masing developer melakukan Unit Test atas kode masing-masing sebelum integrasi dengan developer lainnya menjadi fitur lengkap yang dapat diimplementasikan.

Object Oriented Analysis dan Design

- Adalah pendekatan Analisis dan Design berorientasi Object (OOA) dalam rekayasa perangkat lunak yang memodelkan sistem berdasarkan object dunia nyata.
- **Manfaat:**
 - **Modularitas**: Memecah sistem kompleks menjadi bagian-bagian kecil yang terkelola.
 - **Reuse**: Memudahkan penggunaan ulang komponen atau object yang sama lintas aplikasi.
 - **Maintainability**: Memudahkan pengembangan lain memahami dan merawat kode dikemudian hari
- **Tools**: diagram UML dan OOP

Layered Architecture

- Layered Architecture adalah perwujudan dari OOAD
- Layered Software Design (n-tier architecture atau layered architecture) adalah suatu design pattern yang mengatur suatu software kedalam layer-layer horizontal berbeda-beda.
- Masing-masing layer memiliki tanggung jawab khusus dan hanya berkomunikasi dengan layer diatas ataupun dibawahnya,



Manfaat

- Maintainability: Perubahan pada lapisan Presentation (View) tidak mengubah lapisan Application (Service) ataupun kode database (DAO)
- Independent Testing: Pengujian masing-masing lapisan menggunakan mock data tanpa membutuhkan koneksi database yang sesungguhnya.
- Parallel Development: Masing-masing developer dapat bekerja pada lapisan masing-masing, misalkan developer A mengerjakan bagian View dan developer lain mengerjakan bagian Service secara bersamaan.

Unified Modeling Language

- OOAD menggunakan **Unified Modelling Language (UML)** sebagai alat pemodelan atas rancangan perangkat lunak.
- UML adalah **Bahasa visual atau diagram**
 - **Saat Analisis (OOA):** Menggunakan Use Case Diagram untuk memahami kebutuhan pengguna dan Activity Diagram untuk melihat alur kerja bisnis.
 - **Saat Design (OOD):** Menggunakan Class Diagram untuk merancang struktur kelas dan Sequence Diagram untuk merancang bagaimana objek akan saling berinteraksi

Object Oriented Programming (OOP)

- Adalah suatu paradigma yang mengatur rancangan software terkait data atau object daripada sekedar fungsi dan logika.
- Developer membaca diagram UML yang dibuat System Analyst dan mengimplementasikan kedalam Bahasa pemrograman berorientasi objek seperti Java, Python atau C++
- UML berfungsi sebagai peta petunjuk jalan bagi programmer untuk bagaimana membuat class dan interaksinya menjadi fitur dalam software dan penyimpanannya.

4 Pillar OOP

- **Encapsulation** (enkapsulasi) adalah bundle dari data dan method yang beroperasi pada satu object.
- **Abstraction** (abstraksi) adalah menyembunyikan kompleksitas detail dan menampilkan hanya fitur-fitur penting yang dapat diakses user.
- **Inheritance** (pewarisan) adalah suatu mekanisme yang memperbolehkan suatu subclass mengadopsi atribut dan method dari class yang ada (parent class)
- **Polymorphism** (banyak bentuk) adalah kemampuan objek-objek yang berbeda untuk merespon pemanggilan method dengan cara mereka masing-masing.

Enkapsulasi

- Anda bekerja sebagai Produk Development pada sebuah Perusahaan Perumahan, berdasarkan hasil kesepakatan dengan marketing dan sales, Perusahaan menetapkan standard paket dasar produk Rumah pada saat pemasaran:
 - Atribut (Property)
 - Nama Type:
 - Luas Tanah:
 - Luas Bangunan:
 - Jumlah Kamar Tidur
 - Jumlah Toilet
 - Jumlah Tingkat
 - Kapasitas Listrik
 - Status Rumah
 - Method (Tindakan yang boleh diminta oleh konsumen atas rumah)
 - Ubah Warna Cat Interior()
 - Ubah Letak Kamar ()
 - Ubah Letak Tangga()
 - Ubah Letak Pintu()
 - Ubah Warna Keramik()
 - Tampilkan Spesifikasi Rumah Saat Ini()

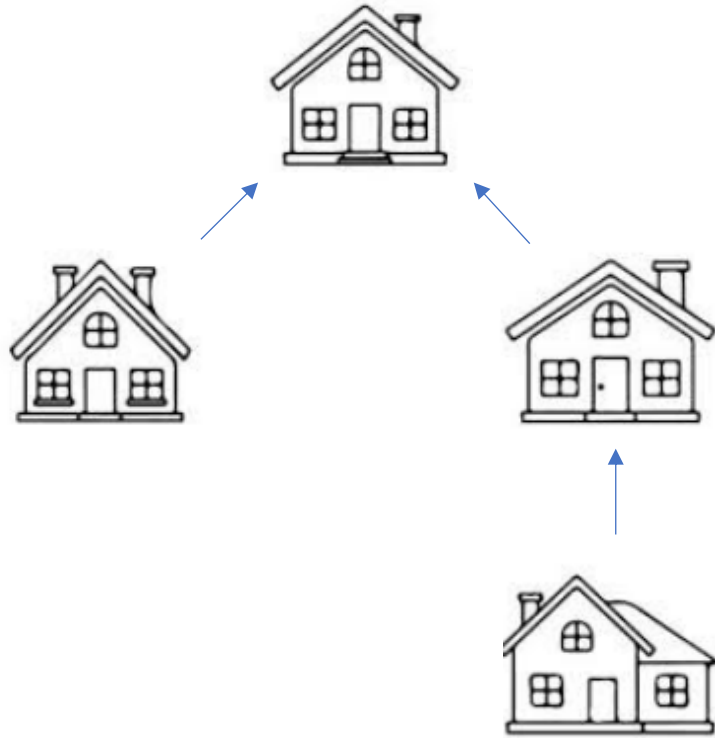
Abstraction

- Rancangan produk rumah tentu saja banyak kerumitan lainnya yang tidak perlu disampaikan pada brosur pemasaran seperti:
 - Atribut
 - Ukuran Pondasi
 - Ukuran Besi Coran
 - Ukuran Pipa Pembuangan
 - Merek Kabel
 - Merek Pipa
 - Method
 - Cor Pondasi()
 - Cor Lantai()
 - Pasang Bata ()
 - Plester Dinding()
 - Buat Atap()
 - Pasang Pintu()
 - dll
- Atribut dan Method detail diatas tidak perlu disampaikan pada saat pemasaran, tetapi dibutuhkan sebagai rancangan teknis pembangunan rumah ataupun pelaksanaan proyek sehingga nantinya rumah dapat menjadi hunian yang baik.

Inheritance

- Perusahaan berhasil sold out pemasaran tahap 1, dan tim produk merancang tahap 2. Berdasarkan trend market dan masukan yang diterima pada penjualan Tahap 1, Perusahaan menetapkan standard paket dasar produk Rumah pada saat pemasaran Tahap 2 yang lebih mewah:
 - Mencakup semua standard paket dasar Tahap 1 ditambah:
 - Atribut:
 - Ukuran Taman Belakang
 - Ukuran Kolam Renang
 - Ukuran Solar Panel
 - Car Port
 - Jumlah CCTV
 - Method
 - Ubah Jenis Tanaman()
 - Ubah Warna Kolam()
 - Ubah Letak CCTV()

Gambaran



Polymorphism

- Pada saat ini telah memasuki tahapan Pembangunan, Perusahaan masih menggunakan kontraktor dan pekerja yang sama, sehingga cara kerja Cor Pondasi() untuk Rumah Tahap1 dan Rumah Tahap 2 dapat berbeda sesuai dengan tipe rumah yang dikerjakan.

Instantiate

- Pembuatan object dari Class
 - Masing-masing objek dapat memiliki atribut(property) yang berbeda
 - Object dari Class yang sama akan memiliki behaviour(method) yang sama

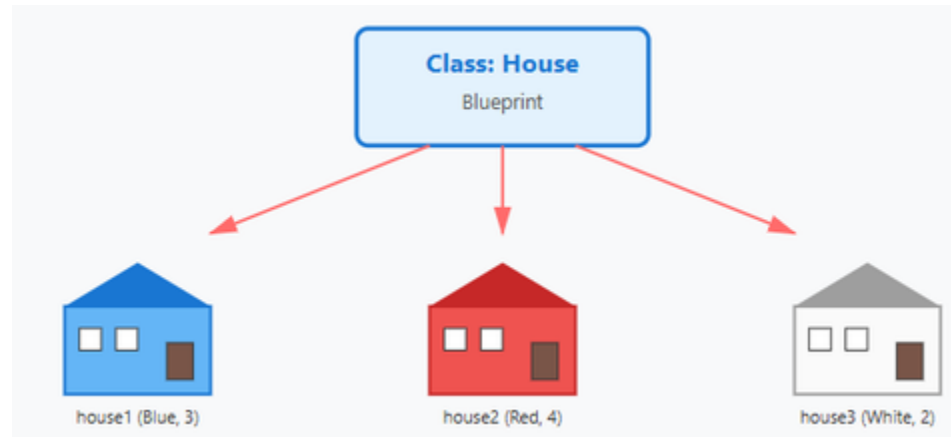
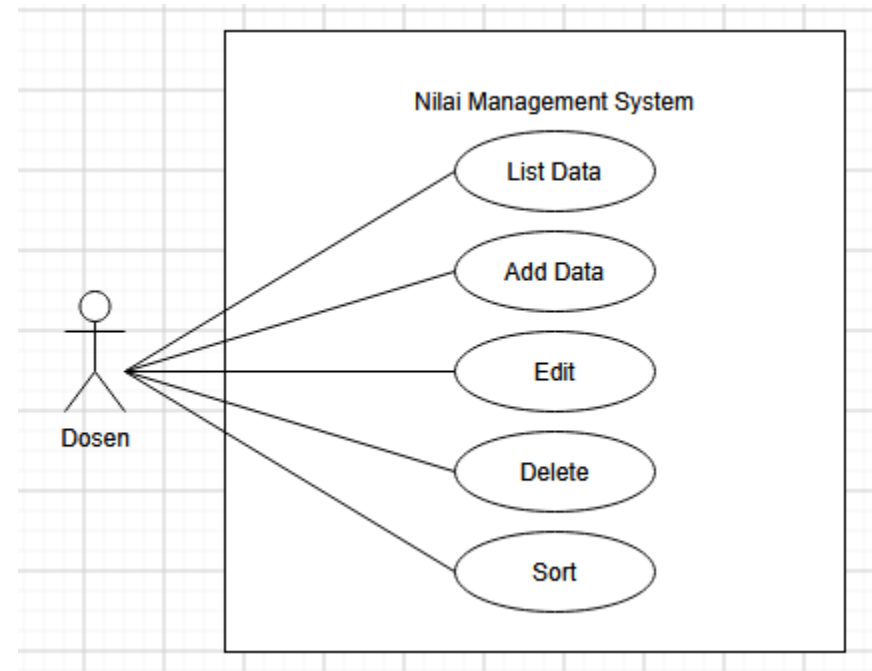


Diagram OOAD

- Ada 4 Diagram paling penting dalam Object Oriented Analysis & Design, yaitu:
 - Use Case Diagram
 - Activity Diagram
 - Class Diagram
 - Sequence Diagram

Use Case Diagram

- Diagram UML yang menunjukkan siapa pengguna sistem dan apa saja yang mereka bisa lakukan (fitur-fitur yang tersedia)
- Diagram ini berfokus pada fungsi sistem dari sudut pandang pemakai (tanpa memedulikan kerumitan kode didalamnya)
- Terdiri dari Actor dan Use Case



Use Case Overview

Use Case ID	Use Case Name	Primary Actor	Description
UC-01	View List Data	User	Display all course grade records stored in the binary file.
UC-02	Add Data	User	Add a new course grade entry with calculated letter grades.
UC-03	Edit Data	User	Update an existing course record using its record number.
UC-04	Delete Data	User	Delete a record and truncate the underlying binary file.
UC-05	Sort Data	User	Sort all records alphabetically by course name using QuickSort.
UC-06	Search Data	User	Find a specific record by course name and display its details.

Use Case Specification

- Merupakan suatu dokumen text yang menunjukkan bagaimana seorang actor berinteraksi dengan sistem untuk mencapai tujuan tertentu.

UC-01: List Data

- Actor: Dosen
- Goal: Menampilkan semua record nilai yang tercatat
- Preconditions: User memilih menu List Data
- Postconditions: Semua record nilai yang tercatat akan ditampilkan
- Main Success Scenario:
 1. User memilih List data pada menu
 2. Aplikasi menampilkan daftar nilai
- Extension (Alternative Flow)
 - 2a. Ketika data kosong: Aplikasi menampilkan “Data Kosong”

UC-02: Add Data

- Actor: Dosen
- Goal: Menambah data nilai baru
- Precondition: User memilih Add Data pada Menu
- Postcondition: Semua data yang input berhasil disimpan
- Main Success Scenario:
 1. User memilih Add Data pada menu
 2. System menanyakan Kode Kuliah
 3. User mengisikan Kode Kuliah
 4. System menanyakan Nilai
 5. User Mengisikan Nilai Kuliah
 6. System menampilkan “Data berhasil disimpan”
 7. Kembali ke Step 2
- Extension (Alternative Flow):
 - 3a. Kode Kuliah Kosong: Use case Add Data selesai
 - 5a. Nilai tidak berada dalam range 0 – 100: System menampilkan “Nilai harus 0-100”
 - 5b. Nilai bukan numerik: System menampilkan “Nilai tidak valid”

UC-03: Edit Data

- Actor: Dosen
- Goal: Mengupdate data nilai yang ada
- Precondition: User memilih Edit Data pada Menu
- Postcondition: Semua data yang diupdate berhasil disimpan
- Main Success Scenario:
 1. User memilih Edit Data pada menu
 2. System menanyakan Nomor Record
 3. User mengisikan Nomor Record
 4. System menampilkan data record
 5. System menanyakan Kode Kuliah diubah menjadi
 6. User mengisikan Kode Kuliah
 7. System menanyakan Nilai Kuliah
 8. User Mengisikan Nilai Kuliah
 9. System menampilkan “Data berhasil diupdate”
 10. User Case Selesai
- Extension (Alternative Flow):
 - 3a. User isi Nomor Record 0: Use Case selesai
 - 6a. Kode Kuliah Kosong: Use Case selesai
 - 7a. Nilai tidak berada dalam range 0 – 100: System menampilkan “Nilai harus 0-100”
 - 7b. Nilai bukan numerik: System menampilkan “Nilai tidak valid”

UC-03: Delete Data

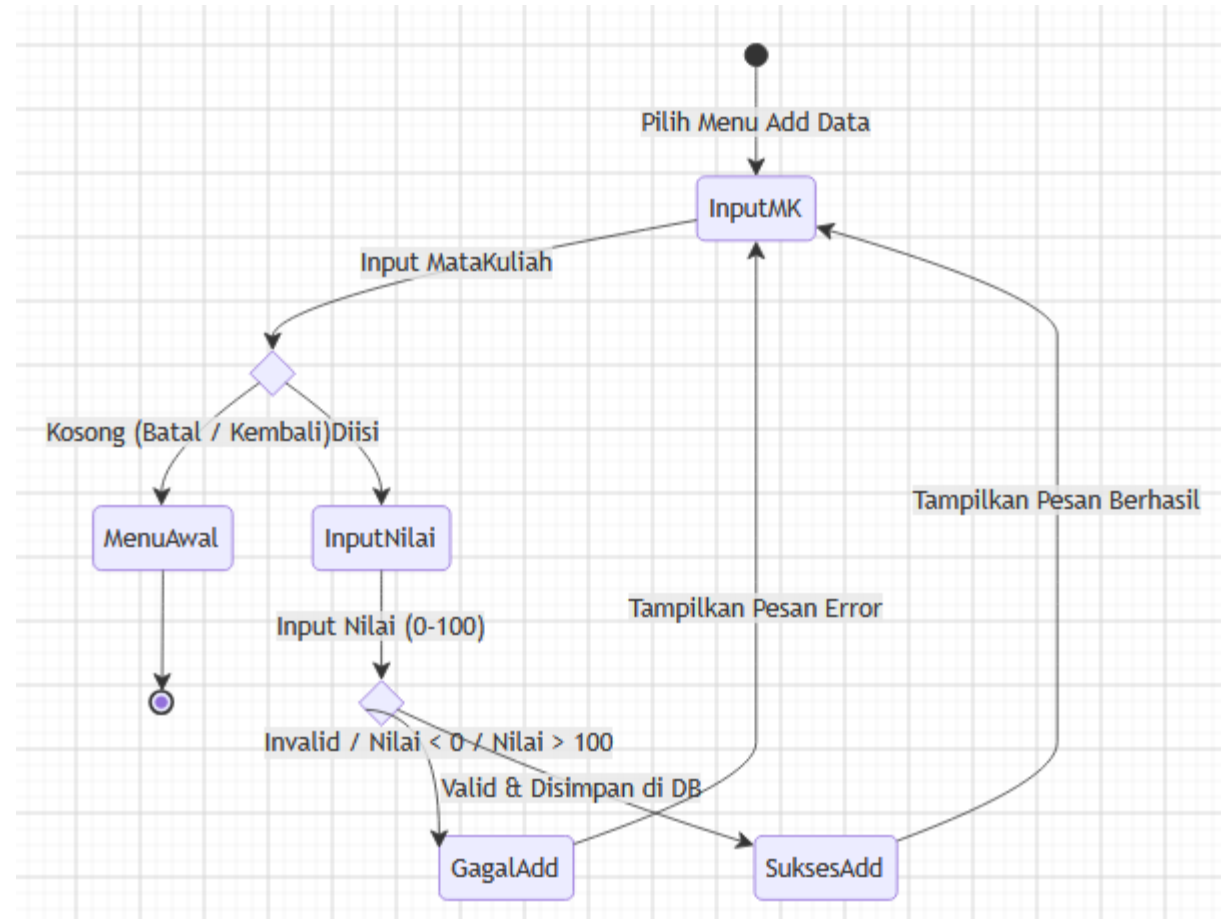
- Actor: Dosen
- Goal: Menghapus data nilai yang ada
- Precondition: User memilih Delete Data pada Menu
- Postcondition: Data dihapus dari penyimpanan
- Main Success Scenario:
 1. User memilih Hapus Data pada menu
 2. System menanyakan Nomor Record
 3. User mengisikan Nomor Record
 4. System menampilkan data record
 5. System menanyakan Yakin hapus data?
 6. User melakukan konfirmasi Ya
 7. System menghapus Record tersebut
 8. System menampilkan "Data berhasil dihapus"
 9. User Case Selesai
- Extension (Alternative Flow):
 - 3a. User isi Nomor Record 0: Use Case selesai
 - 4a. Record tidak ditemukan: System menampilkan "Record tidak ditemukan"

Activity Diagram

- Adalah behavioral diagram yang merepresentasikan dynamic flow, actions dan decision didalam suatu sistem atau process business.

Activity Diagram (Alur Kerja)

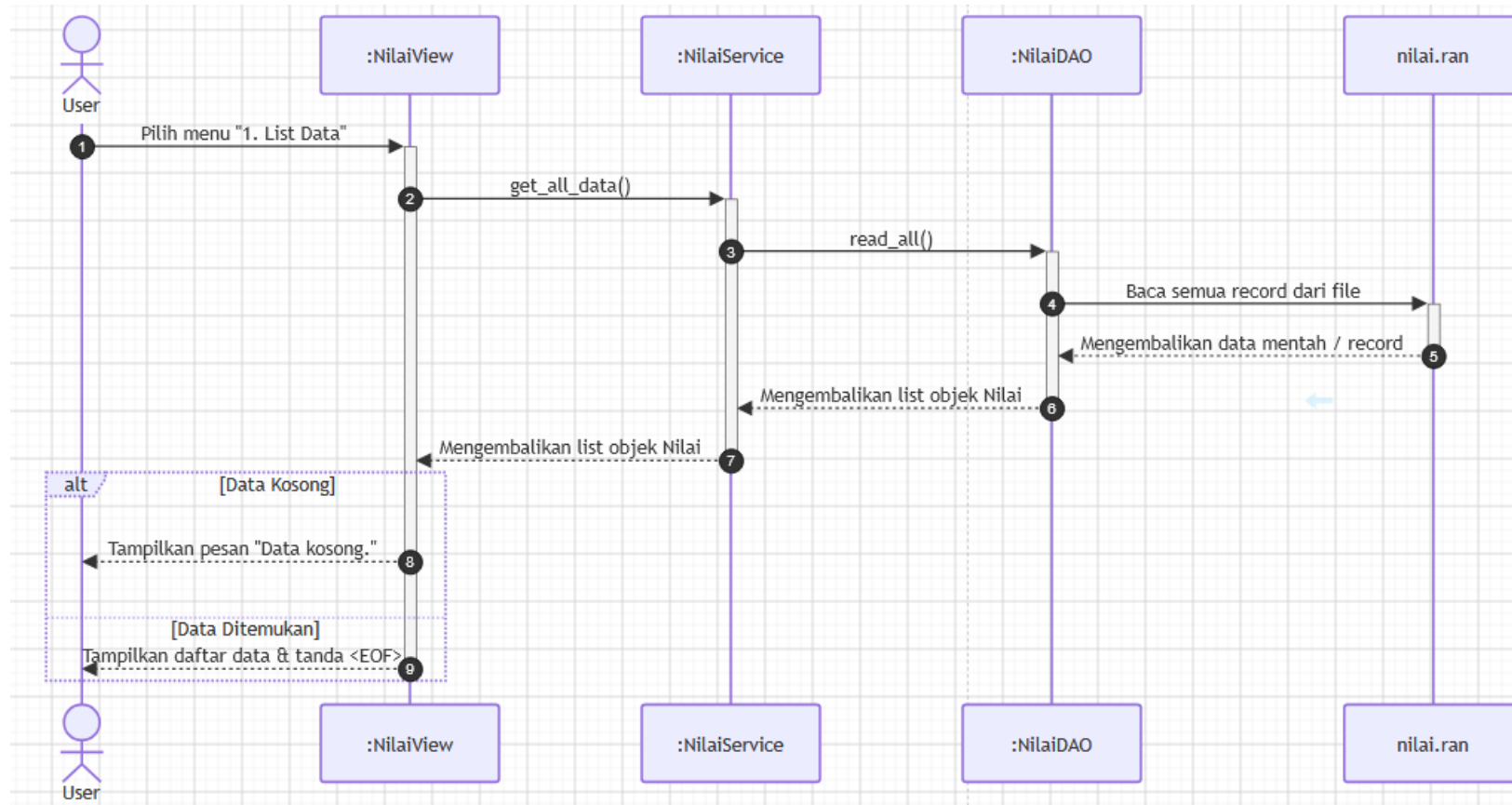
- Diagram ini mirip dengan flowchart, digunakan untuk menggambarkan Langkah-Langkah proses bisnis atau workflow operasional dari sebuah use case.
- Terdiri dari Start, Action, Decision, End.



Sequence Diagram

- Adalah diagram yang menunjukkan diagram interaksi yang secara visual memetakan bagaimana objek dan actor didalam sistem berinteraksi satu dengan lainnya sesuai dengan urutan secara kronologis waktu.
- Objek-objek berinteraksi dengan saling mengirim message/method call untuk menyelesaikan tugas tertentu.

Sequence Diagram UC-01



Penerapan konsep OOAD, dimana pengembangan kode perangkat lunak dibuat menjadi lapisan-lapisan objek yang saling berinteraksi untuk mencapai tujuan dari fitur.

Tugas

- Buka file oop1.py, jalankan dan lakukan refleksi
 - Apakah aplikasi oop1.py sudah menggunakan pendekatan OOP?
 - Apabila setiap modul aplikasi dikembangkan dengan pendekatan seperti oop1.py:
 - Bagaimana masing-masing developer dapat bekerja secara parallel sesuai dengan bidang keahlian?
 - Bagaimana masing-masing modul diuji sesuai dengan bidang penugasan masing-masing developer?
 - Jika terjadi perubahan requirement business, siapa yang harus melakukan pemeliharaan dan pengembangan
- Gambar Use Case Diagram
 - Gunakan appdiagram.net
- Buatlah Use Case Overview
- Gambar Use Case Specification
 - Add Data
 - Edit Data
 - Delete Data
 - Search Data
- Gambar Activity Diagram
 - Add Data
 - Edit Data
 - Delete Data
 - Search Data
- Gambar Sequence Diagram
 - List Data